

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = {
    "Student_ID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Study_Hours": [2, 4, 6, 8, np.nan, 5, 7, 20, 3, 4],
    "Attendance": [85, 90, 88, 95, 80, 78, 92, 110, 75, np.nan],
    "Assignment_Score": [70, 75, 80, 85, 90, 65, 72, 95, 68, 74],
    "Exam_Score": [65, 70, 78, 88, 92, 60, 75, 98, 66, 72],
    "Gender": ["M", "F", "M", "F", "F", "M", "M", "F", "F", "M"]
}

df = pd.DataFrame(data)

print(df)
```

	Student_ID	Study_Hours	Attendance	Assignment_Score	Exam_Score	Gender
0	1	2.0	85.0	70	65	M
1	2	4.0	90.0	75	70	F
2	3	6.0	88.0	80	78	M
3	4	8.0	95.0	85	88	F
4	5	NaN	80.0	90	92	F
5	6	5.0	78.0	65	60	M
6	7	7.0	92.0	72	75	M
7	8	20.0	110.0	95	98	F
8	9	3.0	75.0	68	66	F
9	10	4.0	NaN	74	72	M

```
print(df.isnull().sum())
```

```
Student_ID      0
Study_Hours     1
Attendance      1
Assignment_Score 0
Exam_Score      0
Gender          0
dtype: int64
```

```
df["Study_Hours"].fillna(df["Study_Hours"].mean(), inplace=True)
df["Attendance"].fillna(df["Attendance"].mean(), inplace=True)
```

/tmp/ipykernel_179/2418863968.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c[...] The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df["Study_Hours"].fillna(df["Study_Hours"].mean(), inplace=True)
/tmp/ipykernel_179/2418863968.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c[...] The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]
```

```
df["Attendance"].fillna(df["Attendance"].mean(), inplace=True)
```

```
print(df.isnull().sum())
```

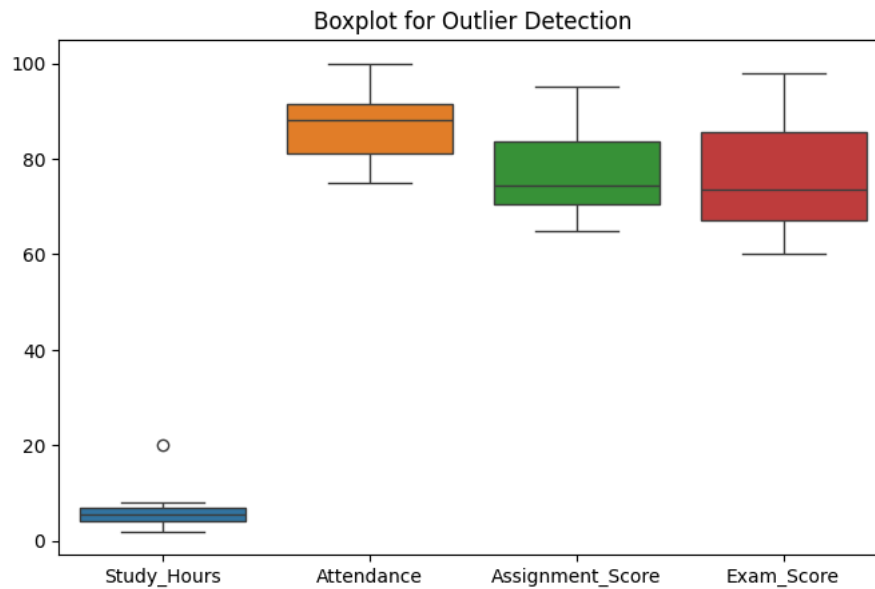
```
Student_ID      0
Study_Hours     0
Attendance      0
Assignment_Score 0
Exam_Score      0
Gender          0
dtype: int64
```

```
print(df[df["Attendance"] > 100])
```

	Student_ID	Study_Hours	Attendance	Assignment_Score	Exam_Score	Gender
7	8	20.0	110.0	95	98	F

```
df.loc[df["Attendance"] > 100, "Attendance"] = 100
```

```
plt.figure(figsize=(8,5))
sns.boxplot(data=df[["Study_Hours", "Attendance", "Assignment_Score", "Exam_Score"]])
plt.title("Boxplot for Outlier Detection")
plt.show()
```



```
Q1 = df["Study_Hours"].quantile(0.25)
Q3 = df["Study_Hours"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
df["Study_Hours"] = np.where(df["Study_Hours"] > upper, upper, df["Study_Hours"])
```

```
df["Exam_Percentage"] = (df["Exam_Score"] / 100) * 100
```

```
df["Log_Study_Hours"] = np.log(df["Study_Hours"])
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
```

```
sns.histplot(df["Study_Hours"], kde=True)
```

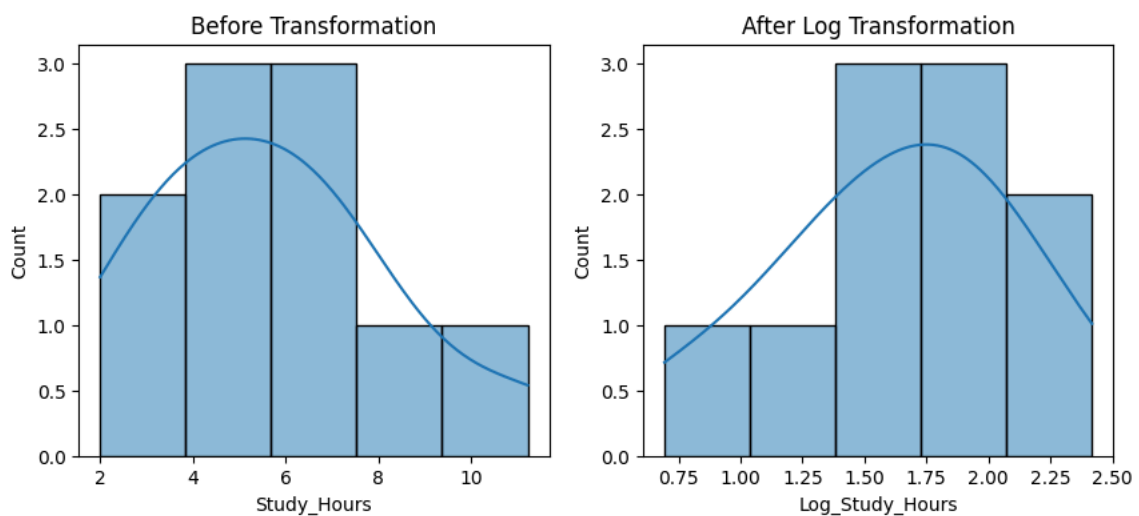
```
plt.title("Before Transformation")
```

```
plt.subplot(1,2,2)
```

```
sns.histplot(df["Log_Study_Hours"], kde=True)
```

```
plt.title("After Log Transformation")
```

```
plt.show()
```



```
print(df)
```

	Student_ID	Study_Hours	Attendance	Assignment_Score	Exam_Score	Gender	\
0	1	2.000000	85.000000	70	65	M	
1	2	4.000000	90.000000	75	70	F	
2	3	6.000000	88.000000	80	78	M	

3	4	8.000000	95.000000	85	88	F
4	5	6.555556	80.000000	90	92	F
5	6	5.000000	78.000000	65	60	M
6	7	7.000000	92.000000	72	75	M
7	8	11.222222	100.000000	95	98	F
8	9	3.000000	75.000000	68	66	F
9	10	4.000000	88.111111	74	72	M

	Exam_Percentage	Log_Study_Hours
0	65.0	0.693147
1	70.0	1.386294
2	78.0	1.791759
3	88.0	2.079442
4	92.0	1.880313
5	60.0	1.609438
6	75.0	1.945910
7	98.0	2.417896
8	66.0	1.098612
9	72.0	1.386294